

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Компьютерная графика»

**ТЕМА: «Формирования различных поверхностей с использованием ее
пространственного разворота и ортогонального проецирования на
плоскость при ее визуализации (выводе на экран дисплея)»**

Студенты гр. 3311

Аршин А.Д.

Баймухамедов

Р.Р.

Пасечный Л.В.

Преподаватель

Колев Г. Ю.

Санкт-Петербург

2025

Цель работы:

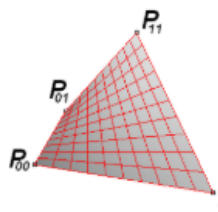
Целью данной работы является разработка программного обеспечения для формирования и визуализации билинейной поверхности на основе четырёх произвольно заданных угловых точек в трёхмерном пространстве и реализации функционала для её интерактивного вращения вокруг осей X и Y.

Задание (вариант 1):

Сформировать билинейную поверхность на основе произвольного задания ее четырех угловых точек. Обеспечить ее поворот относительно осей X и Y.

Теоретическая основа:

Билинейная поверхность порождается четырьмя точками P_{00} , P_{01} и P_{10} и P_{11} следующим образом:



$$(4) \quad Q(u, v) = (1-u, u) \begin{pmatrix} P_{00} & P_{01} \\ P_{10} & P_{11} \end{pmatrix} \begin{pmatrix} 1-v \\ v \end{pmatrix} = P_{00}(1-u)(1-v) + P_{01}(1-u)v + P_{10}u(1-v) + P_{11}uv$$

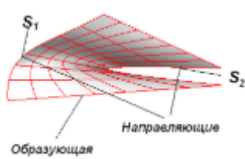
Здесь и далее, по умолчанию предполагается использование нормальной параметризации, т.е. $0 \leq u \leq 1, 0 \leq v \leq 1$.

Параметрические линии билинейной поверхности – отрезки прямых, границы – отрезки , ,

Пример. Программа каркасного изображения билинейной поверхности.

3.3. Линейчатая и бикубическая поверхности Кунса.

Линейчатую поверхность можно рассматривать как поверхность, получающуюся в результате пространственного движения отрезка прямой переменной длины. Сам отрезок в этом случае называется образующей линией, а траектории его концов – направляющими линиями. Можно также считать, что образующая линейно переводит точки одной направляющей линии в точки другой.



Линейчатая поверхность описывается уравнением:

$$(5) \quad Q(u, v) = S_1(u) \cdot (v-1) + S_2(u) \cdot v,$$

где S_1 и S_2 – граничные кривые, направляющие.

Параметрические линии линейчатой поверхности в одном направлении – отрезки, в другом – кривые, линейно трансформирующиеся от одной направляющей к другой. Билинейная поверхность является частным случаем линейчатой поверхности.

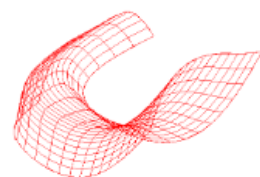
Для целей геометрического моделирования было бы более удобно задавать все четыре границы порции поверхности в виде кривых

$$(6) \quad \begin{aligned} Q(u, 0) &= S_1(u), & Q(u, 1) &= S_2(u) \\ Q(0, v) &= S_3(v), & Q(1, v) &= S_4(v) \end{aligned}$$

Для "смешивания" граничных кривых можно использовать линейную интерполяцию:

$$(7) \quad Q(u, v) = (1-u, u) \begin{pmatrix} Q(0, v) \\ Q(1, v) \end{pmatrix} + (Q(u, 0) - Q(u, 1)) \begin{pmatrix} 1-v \\ v \end{pmatrix} - (1-u, u) \begin{pmatrix} Q(0, 0) & Q(0, 1) \\ Q(1, 0) & Q(1, 1) \end{pmatrix} \begin{pmatrix} 1-v \\ v \end{pmatrix}$$

Последний член вычитается, так как при сложении двух первых слагаемых, задающих интерполяцию в u и v направлениях возникает "лишняя" билинейная поверхность. Это происходит из-за того, что каждая из "угловых" точек принадлежит паре граничных кривых, например, $Q(0, 0)$ принадлежит одновременно кривым $Q(u, 0)$ и $Q(0, v)$. Таким образом, при суммировании двух первых членов влияние каждой угловой точки удваивается.



Линейчатая поверхность является частным случаем линейной поверхности Кунса.

Бикубический лоскут (bicubic patch) - это поверхность, определяемая полиномиальным уравнением третьего порядка по параметрам u и v .

$$P(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij} u^i v^j \quad (0 \leq u \leq 1, 0 \leq v \leq 1).$$

13. Конические сечения кривые. Кривая Безье, b-сплайн

$B(t) = \sum_{i=0}^n P_i b_{i,n}(t)$, $0 < t < 1$ $B(t) = (1-t)P_0 + tP_1$ $t \in [0, 1]$ Кривая Безье — параметрическая кривая, задаваемая выражением: Линейные кривые При $n = 1$ кривая представляет собой отрезок прямой линии, опорные точки P_0 и P_1 определяют его начало и конец. Кривая задаётся уравнением:

Квадратичная кривая Безье ($n = 2$) задаётся 3-мя опорными точками: P_0 , P_1 и P_2 .

$$B(t) = (1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2, \quad t \in [0, 1]$$

Квадратичные кривые Безье в составе сплайнов используются для описания формы символов в шрифтах TrueType и в SWF файлах.

Кубические кривые В параметрической форме кубическая кривая Безье ($n = 3$) описывается следующим уравнением:

$$B(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t)P_2 + t^3 P_3, \quad t \in [0, 1]$$

Благодаря простоте задания и манипуляции, кривые Безье нашли широкое применение в компьютерной графике для моделирования гладких линий. Кривая целиком лежит в выпуклой оболочке своих опорных точек. Это свойство кривых Безье с одной стороны значительно облегчает задачу нахождения точек пересечения кривых, а с другой стороны позволяет осуществлять интуитивно понятное управление параметрами кривой в графическом интерфейсе с помощью её опорных точек.

В вычислительной математике В-сплайном называют сплайн-функцию, имеющую наименьший носитель для заданной степени, порядка гладкости и разбиения области определения. Фундаментальная теорема устанавливает, что любая сплайн-функция для заданной степени, гладкости и области определения может быть представлена как линейная комбинация В-сплайнов той же степени и гладкости на той же области определения. Термин В-сплайн был введён И. Шёнбергом и является сокращением от словосочетания «базисный сплайн». [2] В-сплайны могут быть вычислены с помощью алгоритма де Бора, обладающего устойчивостью. В системах автоматизированного проектирования и компьютерной графике термин В-сплайн часто описывает сплайн-кривую, которая задана сплайн-функциями, выраженными линейными комбинациями В-сплайнов.

$$P(u) = \sum_{i=0}^n P_i N_{i,k}(u) \quad (t_{k-1} \leq u \leq t_{k+1}),$$

$$N_{i,k}(u) = \frac{(u - t_i)N_{i,k-1}(u)}{t_{i+k-1} - t_i} + \frac{(t_{i+k} - u)N_{i+1,k-1}(u)}{t_{i+k} - t_{i+1}},$$

$$N_{i,1}(u) = \begin{cases} 1 & t_i \leq u \leq t_{i+1}; \\ 0 & \text{в противном случае.} \end{cases}$$

Рис. 2

Выполнение лабораторной работы

В качестве языка программирования для выполнения данной лабораторной работы выберем Python. Для быстрого запуска достаточно иметь установленный Python 3 (модули tkinter и math, которые входят в стандартную библиотеку). Демонстрация (<https://disk.yandex.ru/i/rlxIDgdBSUqsjw>)

Интерфейс созданного приложения:

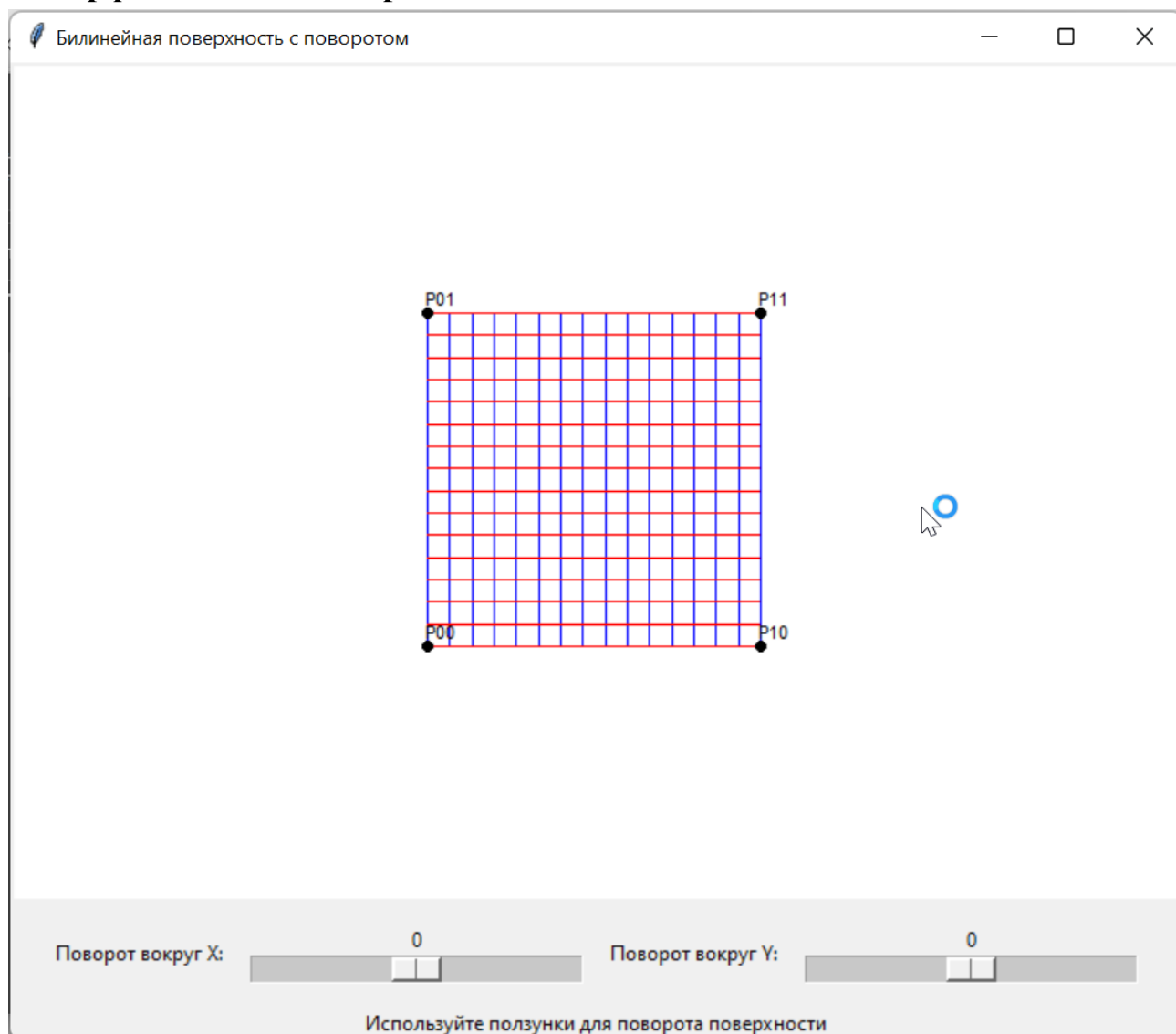


Рис. 3 Готовое приложение

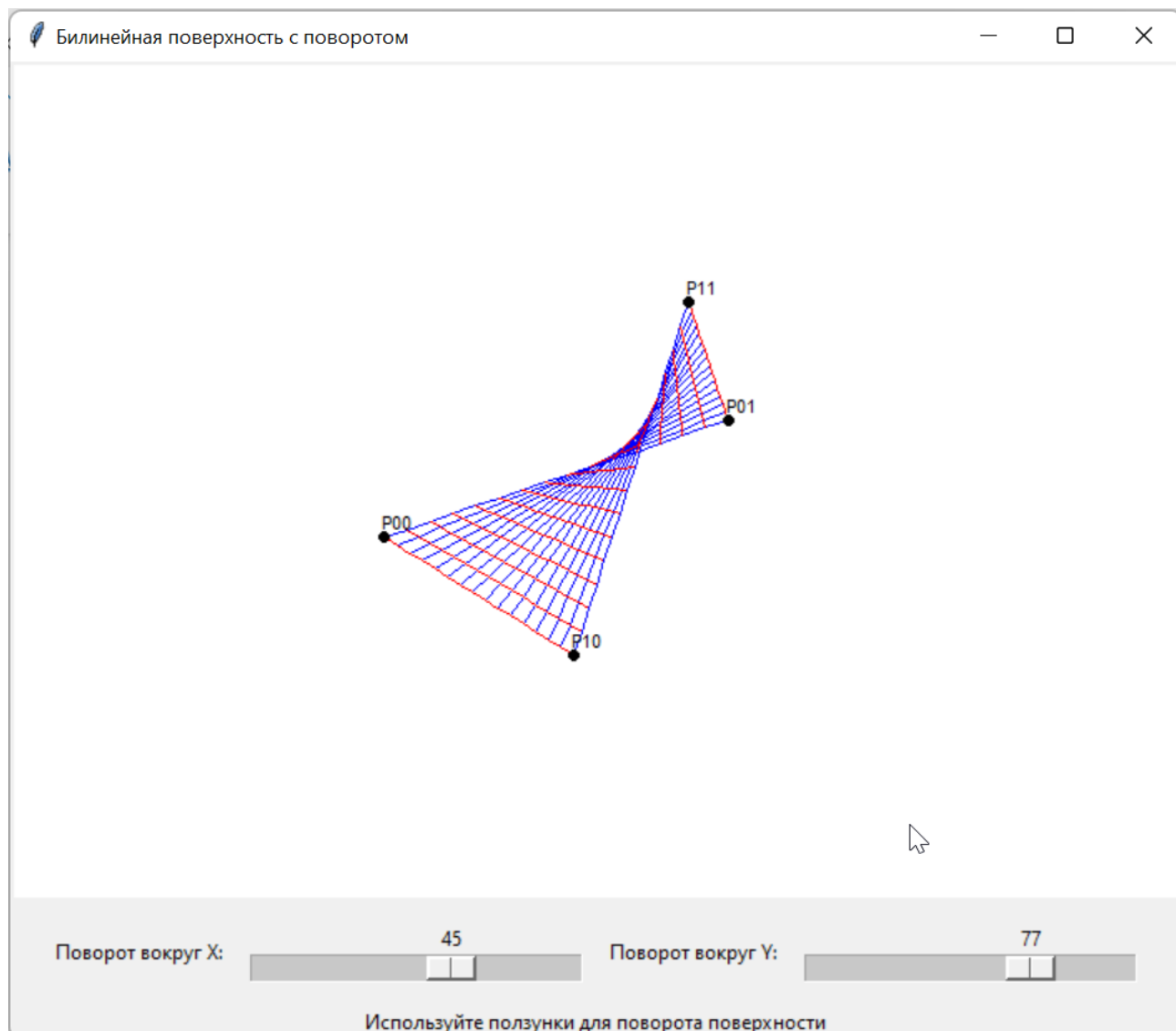


Рис. 4 показ работы с поворотом

Вывод:

В ходе выполнения работы была успешно достигнута поставленная цель: разработана программа, которая формирует билинейную поверхность и обеспечивает её поворот вокруг осей X и Y.

Можно сделать следующие заключительные выводы:

1. Реализация билинейной поверхности: Билинейная поверхность была корректно сформирована на основе четырёх угловых точек. Аппарат билинейной интерполяции доказал свою эффективность для построения поверхностей, которые естественным образом "натягиваются" на заданные опорные точки. Полученная поверхность представляет собой гладкий переход между угловыми точками и является частным случаем поверхности Кунса.
2. Реализация поворота: Алгоритмы поворота с использованием матриц преобразований показали свою точность и надежность. Поворот вокруг осей X и Y реализован корректно, что позволяет всесторонне изучать геометрию созданной поверхности. Операция поворота является

линейным преобразованием и применяется к каждой вычисленной точке поверхности, что гарантирует сохранение её формы.

3. Эффективность подхода: Использование параметрического задания поверхности и матриц поворота является классическим и эффективным методом в компьютерной графике. Данный подход позволяет легко комбинировать различные преобразования (например, последовательные повороты вокруг разных осей) и интегрировать в более сложные графические конвейеры.
4. Практическая значимость: Выполненная работа является фундаментальной для понимания принципов геометрического моделирования. Освоенные навыки могут быть применены для работы с более сложными объектами, такими как B-сплайновые или NURBS-поверхности, которые часто представляются в виде сетки билинейных или бикубических патчей.

Приложение 1: Листинг программы

```
import tkinter as tk
import math

# Класс для управления всей программой
class BilinearSurfaceApp:
    # root основное окно tkinter
    # устанавливаем заголовок окна и задаем размеры
    # создаем холст с помощью canvas и задаем белый фон
    def __init__(self, root):
        self.root = root
        self.root.title("Билинейная поверхность с поворотом")
        self.canvas_width = 700
        self.canvas_height = 500

        self.canvas = tk.Canvas(root, width=self.canvas_width,
height=self.canvas_height, bg="white")
        self.canvas.pack()

        # Исходные 4 угловые точки в 3D: (x, y, z)
        # Расположим их как четырёхугольник в пространстве
        # они не лежат в одной плоскости поэтому поверхность будет изогнутой
        self.control_points_3d = [
            [-100, -100, -50], # P00 нижний левый угол
            [100, -100, 50],    # P10 нижний правый угол
            [-100, 100, 50],    # P01 верхний левый угол
            [100, 100, -50]     # P11 верхний правый угол
        ]

        # инициализация углов поворота
        self.angle_x = 0
        self.angle_y = 0

        # Интерфейс для создания ползунков которые будет находиться в рамке frame
        control_frame = tk.Frame(root)
        # Отступы как padding
```

```

control_frame.pack(pady=10)

# Поворот вокруг X в диапазоне от -180 до 180 градусов
# при изменении вызываем метод update_from_sliders
# ползунок с помощью Scale
tk.Label(control_frame, text="Поворот вокруг X:").grid(row=0, column=0,
padx=5)
    self.slider_x = tk.Scale(control_frame, from_=-180, to=180,
orient=tk.HORIZONTAL, length=200,
                                command=self.update_from_sliders)
    self.slider_x.set(0)
    self.slider_x.grid(row=0, column=1, padx=5)

# Аналогичный поворот вокруг Y
tk.Label(control_frame, text="Поворот вокруг Y:").grid(row=0, column=2,
padx=5)
    self.slider_y = tk.Scale(control_frame, from_=-180, to=180,
orient=tk.HORIZONTAL, length=200,
                                command=self.update_from_sliders)
    self.slider_y.set(0)
    self.slider_y.grid(row=0, column=3, padx=5)

tk.Label(root, text="Используйте ползунки для поворота поверхности").pack()

# рисуем поверхность при запуске
self.draw_all()

# Метод поворота точки в 3-д
def rotate_point_3d(self, point, ang_x, ang_y):
    x, y, z = point

    # Поворот вокруг X
    # преобразует углы в радианы
    # игрек и зед меняются, а икс остается
    rad_x = math.radians(ang_x)
    y1 = y * math.cos(rad_x) - z * math.sin(rad_x)
    z1 = y * math.sin(rad_x) + z * math.cos(rad_x)
    y, z = y1, z1

    # Поворот вокруг Y
    # меняются икс и зед, а игрек остается
    rad_y = math.radians(ang_y)
    x2 = x * math.cos(rad_y) + z * math.sin(rad_y)
    z2 = -x * math.sin(rad_y) + z * math.cos(rad_y)
    x, z = x2, z2

    # возвращаем новую 3-д координату после поворота
    return [x, y, z]

# билинейная интерполяция. вычисляем точку на билинейной поверхности
def bilinear_interpolation(self, u, v, p00, p10, p01, p11):

    # u, v — параметры от 0 до 1, задающие положение на поверхности

```

```

        # p00, p10, p01, p11 – угловые точки (уже повёрнутые)
        # формула билинейно интерполяции выглядит так :  $P(u,v) = (1-u)(1-v)P_{00} +$ 
 $u(1-v)P_{10} + (1-u)vP_{01} + uvP_{11}$ 
        x = (1 - u) * (1 - v) * p00[0] + u * (1 - v) * p10[0] + (1 - u) * v *
p01[0] + u * v * p11[0]
        y = (1 - u) * (1 - v) * p00[1] + u * (1 - v) * p10[1] + (1 - u) * v *
p01[1] + u * v * p11[1]
        z = (1 - u) * (1 - v) * p00[2] + u * (1 - v) * p10[2] + (1 - u) * v *
p01[2] + u * v * p11[2]
        # 3-д точка на поверхности
        return [x, y, z]

# метод проекции из 3д --> 2д
def project_to_2d(self, point_3d):
    # Ортогональная проекция: просто отбрасываем Z, смещаем в центр холст
    x, y, z = point_3d
    cx = self.canvas_width // 2
    cy = self.canvas_height // 2
    scale = 1.0 # добавляем масштабирование
    return (cx + x * scale, cy - y * scale) # инвертируем Y для экранной
системы

# Генерация сетки точек нашей поверхности
def generate_surface_points(self, steps=20):
    # генерация сетки точек после поворота
    p00, p10, p01, p11 = self.control_points_3d

    # Поворачиваем все 4 угловые точки
    p00_r = self.rotate_point_3d(p00, self.angle_x, self.angle_y)
    p10_r = self.rotate_point_3d(p10, self.angle_x, self.angle_y)
    p01_r = self.rotate_point_3d(p01, self.angle_x, self.angle_y)
    p11_r = self.rotate_point_3d(p11, self.angle_x, self.angle_y)

    # для всех комбинаций (u, v) вычисляем 3-д точку --> проецируем в 2-д
    surface_2d = []
    for i in range(steps + 1):
        u = i / steps
        row = []
        for j in range(steps + 1):
            v = j / steps
            pt_3d = self.bilinear_interpolation(u, v, p00_r, p10_r, p01_r,
p11_r)
            pt_2d = self.project_to_2d(pt_3d)
            row.append(pt_2d)
        surface_2d.append(row)
    # получаем двумерный список 2-д координат
    return surface_2d

# Отрисовываем все изменения
def draw_all(self):
    # очищаем экран перед прорисовкой
    self.canvas.delete("all")

# Генерация повернутой поверхности в размере 15 на 15

```



```

        grid = self.generate_surface_points(steps=15)

        # Рисуем линии по U (горизонтальные)
        # Каждая строка (row) – это линия при фиксированном u (меняется v)
        # flat – "сплющиваем" список точек вида [(x1,y1), (x2,y2), ...] --> [x1,
y1, x2, y2, ...], что требуется create_line
        for row in grid:
            flat = [coord for pt in row for coord in pt]
            self.canvas.create_line(flat, fill="blue", width=1)

        # Рисуем линии по V (вертикальные) аналогично но по столбцам
        # Горизонтальные линии-синие, а вертикальные-красные
        for col_idx in range(len(grid[0])):
            col_points = [grid[row_idx][col_idx] for row_idx in range(len(grid))]
            flat = [coord for pt in col_points for coord in pt]
            self.canvas.create_line(flat, fill="red", width=1)

        # Подписи углов
        # рисуем черные круги в углах и подписываем
        corners_3d = [self.rotate_point_3d(p, self.angle_x, self.angle_y) for p in
self.control_points_3d]
        corners_2d = [self.project_to_2d(p) for p in corners_3d]
        labels = ["P00", "P10", "P01", "P11"]
        for (x, y), label in zip(corners_2d, labels):
            self.canvas.create_oval(x-3, y-3, x+3, y+3, fill="black")
            self.canvas.create_text(x+8, y-8, text=label, font=("Arial", 8))

        # Обновление экрана при движении ползунков
        # считываем текущие значения углов и перерисовывем все заново
        def update_from_sliders(self, val=None):
            self.angle_x = self.slider_x.get()
            self.angle_y = self.slider_y.get()
            self.draw_all()

        # запуск программы
        if __name__ == "__main__":
            root = tk.Tk()
            app = BilinearSurfaceApp(root)
            root.mainloop()

```